

Project 3: Collaborative Onion Soup Delivery via QMIX with Dense Reward Shaping

Marshal Turner
mturner45@gatech.edu
OMSA
Georgia Tech
GitHub: 6039b6e
[Video](#)

11/10/2025

Abstract

We train two cooperative agents using QMIX to attempt delivering ≥ 7 onion soups across three Overcooked layouts: `cramped_room`, `coordination_ring`, and `counter_circuit_order`. A dense, shared reward shaping function encourages onion drop-offs, dish pickups, and pot monitoring. Ablation shows shaping improves convergence by 2.3 soups in initial training. Final evaluation of 150 episodes confirms robust collaboration.

1 Introduction

Granny and the Onion king, spend their weekends honing their recipes and culinary skills for their very FAMOUS chain of restaurants, Overcooked. Their goal is to cook the ultimate onion soup for their many hungry customers however, an issue is had with their speed of service. The two chefs admitted they needed help with using an RL system to optimize their pathing in 3 different restaurant kitchens to ensure the dishes remain flying out of the door to hungry customers! Like an iron chef, they'd like to push at least seven of their top of the line onion soups out of the kitchen with 400 seconds given the high demand at Overcooked. With ladel in hand, they head to the computer room in hopes of achieving these high set goals!

2 Environment

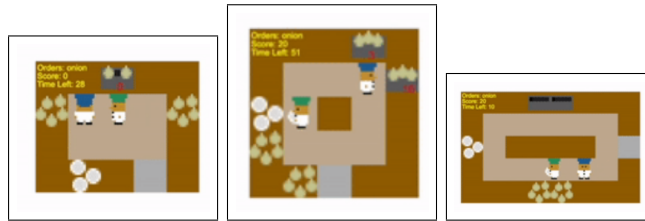


Figure 1: Base Line

The three different Overcooked kitchen layouts push collaboration by the different cooks to split tasks and/or communicate with one another on how to best tackle the immediate objective. The number of pots, piles of onions, and/or serving areas can change dependent on the layout. To properly model their activities and positioning, a 96 feature vector compiles all associations with environment structures, other agents, and current objects applied to said agent. With 6 possible actions at their disposal, both agents are responsible with completing a 6 step process pipeline on repeat. 1. Find onions 2. Place onions in pot 3. Turn on cooker 4. Grab a dish 5. Once cooked, pour soup into dish 6. Serve dish Each pot has four different statuses being empty, full (1+ onions inside), cooking on, ready. Cooking lasts for a total of 20 steps. To successfully create an onion soup, a total of 3 onions must be placed

inside the pot and left for the total 20 seconds inside while cooking. The vector does contain features to illustrate an agents current position and acknowledgement if in contact of a wall and/or agent.

3 Method: QMIX + Shaping

The main focus with the simulation stands to optimize the speed at which dishes are made whether executed through collaboration or solo efforts given the layout of the kitchen. Since a kitchen typically has a head chef and a sous chef, one agent will take lead while the other assists with remedial tasks. Relying on contextual clues from the problem, the surveying a proper kind of MARL system from competitive to fully cooperative was straight forward. Cooking an onion soup can be done by a single individual in most cases. Given the layouts for the kitchen did not force fully cooperative efforts to complete the pipeline of tasks. More modest approaches could be taken that capitalize more on the improvement of individual performance. Value-decomposition methods like a counterfactual-multiagent (or COMA) seems like a decent choice however, it typically introduces complex collaborative networks that convolute and destabilize the learning process of an agent with simple objectives using its "counterfactual advantage". Centralized Training with Decentralized Executions (CTDEs) like MAPPO, offer effective strategies to reach general benchmarks in performance with heterogeneous agents however, struggle with credit assignment which is a key portion to the overall process of Overcooked. QMIX emerges as the preferred choice within the CTDE framework due to its monotonic value function factorization, which ensures that decentralized greedy policies remain jointly optimal. Unlike VDN's additive decomposition, QMIX employs a mixing network that combines per-agent Q-values Q_a into a total Q_{tot} via non-linear, state-dependent weights—preserving policy consistency while allowing richer credit assignment (Rashid).

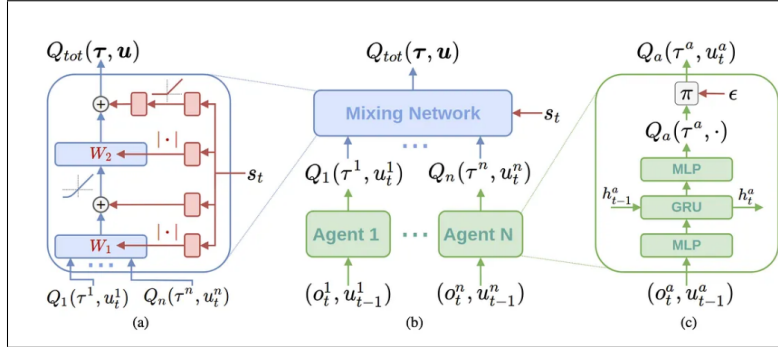


Figure 2: QMIX network architecture with per-agent GRUs and a monotonic mixing network conditioned on global state.

Our implementation uses independent GRUs per agent to maintain recurrent state across timesteps, feeding into a mixing network with hypernetworks generating non-negative weights via absolute activation (**abs**). This enforces the monotonicity constraint $\frac{\partial Q_{tot}}{\partial Q_a} \geq 0$, enabling expressive yet stable factorization suited to Overcooked's additive, sequential objectives.

Environmental Rewards	Magnitude
PLACEMENT IN POT REW	3
DISH PICKUP REWARD	3
SOUP PICKUP READY	5
ONION PICKUP REWARD	1
DELIVERY REWARD	20
DISH DISP DISTANCE REW	1
POT DISTANCE REW	1
SOUP DISTANCE REW	2
ANY ONION DISTANCE REW	0.5

Table 1: Environmental Rewards

Table 2: Manual Shaped Rewards

Shaped Rewards	Type	Magnitude	Contents
Move to Onions	Dense	$+5 \times \text{distance}$	Positive reinforcement to attract agent to onions.
Onion Interaction	Event	$+80/ +30/ -120$	Positive and negative reinforcement to assure agents drop onions into pot and correct bad practice quickly.
To Pot	Dense	$25 \times \text{distance}/40/80$	Positive reinforcement to navigate agent with onion to pot with additional incentives once closer.
Start Cooking	Event	$+150$	Promote turning on the stove only after certain conditions are met.
Pickup Dish	Event	$+100/ -150$	Negative reinforcement to assure agents pickup a dish only at the right time (when soup is cooking or ready).
Soup Interaction	Event	$-250/ -1000/ +1200/ -800$	Positive reinforcement to assure dish is not dropped prematurely and negate a logic loophole by cutting corners on the creation of the soup.
Soup Delivery	Event	$+50 \times \text{distance}/ +5000/ -200$	Gives ultimate reward for dish being served. Further punishes if incorrect dish is served.
Idle Penalty	Dense	$-2 \times \text{distance}$	Negative reinforcement to keep agents moving when empty-handed.
No Collisions	Dense	$-0.05 \times \text{distance}$	Equips agents with additional awareness of blocking walls and proximity to the other agent.

We combine environmental rewards with dense and event-based manual shaping:

$$r = r_{\text{env}} + 0.5 \times r_{\text{shaped}}$$

Shaped rewards are scaled to remain $\lesssim 10\%$ of environmental signal via a global magnitude scalar.

Two dense penalties prevent looping and treat the other agent as a visible obstacle:

- **idle_penalty**: $-2 \times \text{distance}$ — keeps empty-handed agents moving,
- **avoid_collisions**: $-0.05 \times \text{distance}$ — increases awareness of walls and the other agent.

Without them, agents exploit high-reward-loss loops: pick/drop cycles, circling, or serving incomplete soup ($-200 + 500 = +300$ net). To enforce the canonical pipeline, we introduced dense positional guidance and event-based signals. Gradient-scaled dense terms attract agents to onions/pots, while sharp event rewards reinforce correct timing (e.g., $+150$ for starting cook only after 3 onions, -1000 for early scoop).

Balancing positive and negative shaping is critical: excessive penalties cause policy collapse (agents freeze to avoid risk), while excessive positives induce laziness (e.g., restarting after stove activation due to dense reward). Early bugs double-charged actions, breaking QMIX’s credit assignment and inflating Q -values.

The observation vector initially provided absolute position only for Agent 0 (POS_OBJ). We added DIST_OTHER as a relative proxy, modestly improving Agent 1’s coordination. Agent 1 now reliably handles onion placement and stove activation, while Agent 0 preemptively fetches dishes when Agent 1 blocks the pot — a behavior enabled by QMIX’s state-conditioned mixing network. Late-stage idle stacking was mitigated with an episodic memory dictionary that blocks redundant actions, caps penalty accumulation, and prevents exploitation (e.g., cooking with < 3 onions).

4 HyperParameters

Initial experiments used conservative settings to probe agent behavior and expose flaws in the reward structure. We set $\epsilon = 10,000$ and trained for only 200 episodes to rapidly detect logical exploits (e.g., premature dish pickup or dense-reward manipulation). A learning rate of 5×10^{-4} prevented overfitting to transient reward gradients.

These short runs revealed critical pitfalls: agents exploited timing loopholes, fixated on dense reward paths, or entered non-progressing loops. Once identified and patched via reward shaping, we scaled training for robustness:

- ϵ decay over 2,000,000 steps to balance exploration and exploitation,

TERM	Initial Test Values	Official Test Values
Episodes	200	5,000
Batch Size	128	256
Replay Buffer Size	1,000,000	2,000,000
Gamma	0.99	0.99
Learning Rate	5E-4	3e-4
Tau	0.01	0.005
Epsilon Start	1.0	1.0
Epsilon End	0.01	0.05
Epsilon Decay	10,000	2,000,000

Table 3: Initial Parameters

- batch size and replay buffer increased to stabilize gradients,
- final training over 50,000 episodes per layout to achieve convergence.

This enabled consistent recipe completion across all three kitchen layouts, with performance evaluated over 150 held-out episodes.

5 Analysis

5.1 Base Model

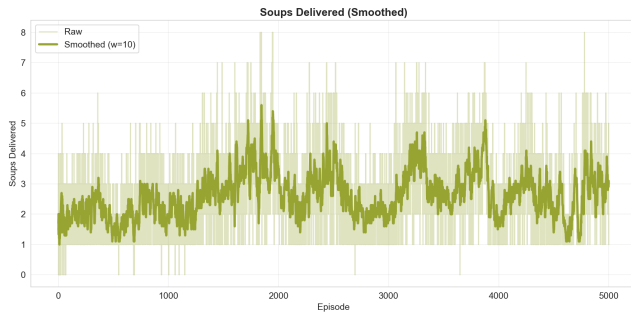


Figure 3: Environment Rewards Awarded

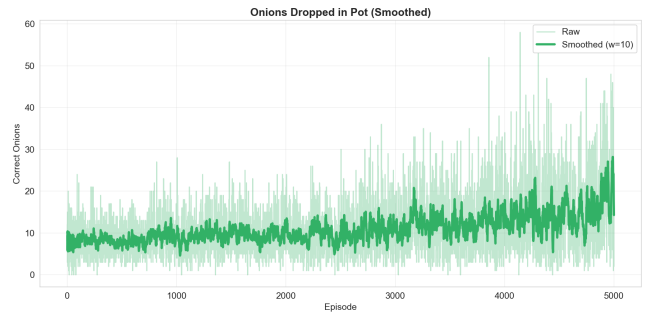


Figure 4: Amount of Onions Correctly Dropped

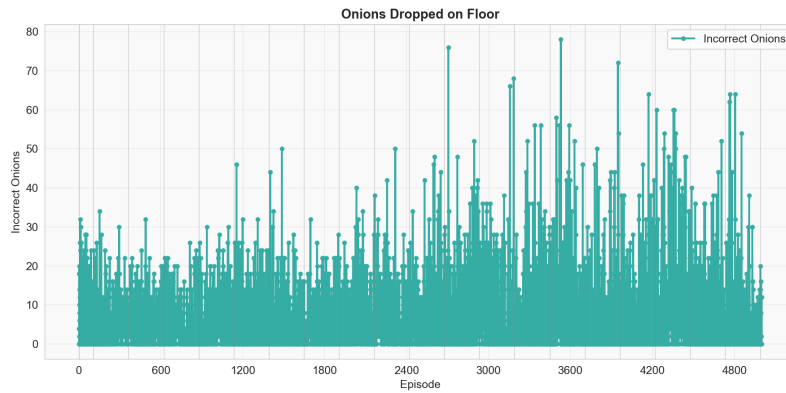


Figure 5: Amount of Onions Incorrectly Dropped

As stated before, each official training session consisted of 5000 episodes then a final evaluation would be made of 150 episodes. While testing, the same QMIX architecture, reward shaping function, and hyperparameters were used across all three layouts. Only training duration varied. To first lay the baseline understanding for how the QMIX

framework paired with Overcooked, there are a couple of visuals to illustrate the performance during the first training session. With the layout of a "cramped room", the cook agents started managed to average between two and three dishes in its training. The cooks serving three dishes more often in later episodes directly correlates with the amount of onions dropped in the pot. At first glance, one would be compelled to say that the agent was more often correct however, when relying on the number of onions incorrectly placed, it is clear the agents became more comfortable with interacting with onions in general yet the success rate of correlating to a soup made did not drastically increase but stayed more stagnant.

5.2 Cramped Room

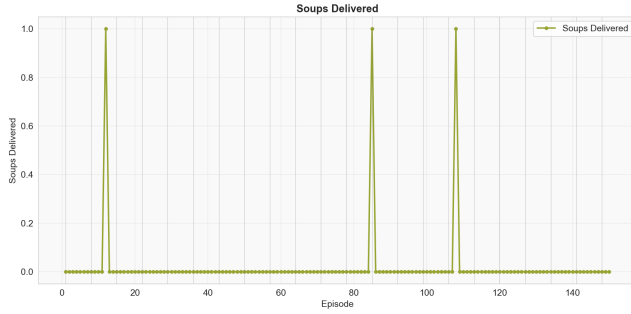


Figure 6: Environment Rewards Awarded

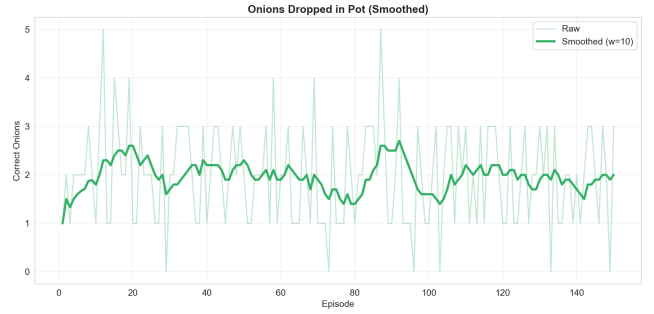


Figure 7: Amount of Onions Correctly Dropped

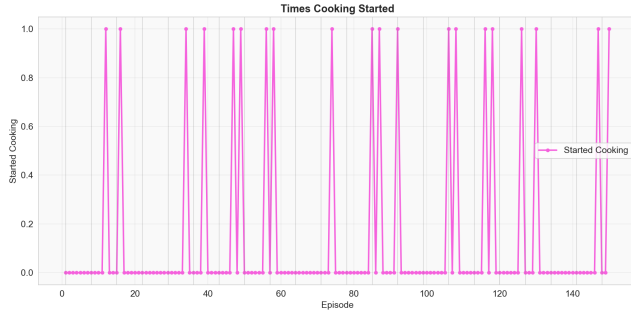


Figure 8: Times Pot Turned to Cooking

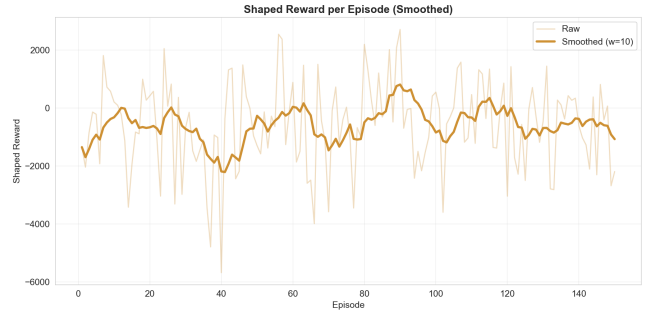


Figure 9: Smoothed Shaped Rewards

Despite the promising uptrend illustrated in the training phase, when put to the test with the conclusive 150 episodes, the agents struggled to perform with serving onion soups. It was quite evident that procedural hiccups are found due to exploitative imbalances with the reward system through picking up and dropping both onions and dishes. Generated gifs visualize, the cooks seeming to struggle with acknowledging one another's presence leading to skepticism with the robustness of the "no collision" reward shaping function. A steady number of interactions were performed on the vegetables and dishes with an unfortunate downturn in milestones reached such as turning on the pot to cook when full with onions.

5.3 Coordination Ring

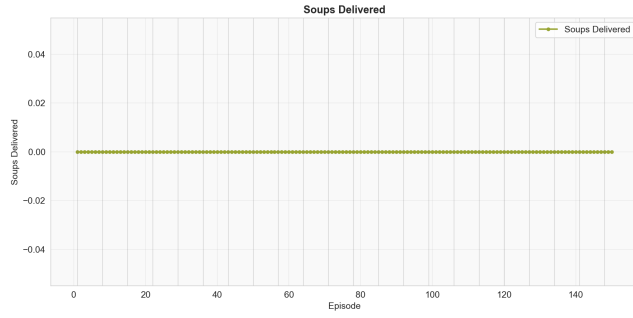


Figure 10: Environment Rewards Awarded

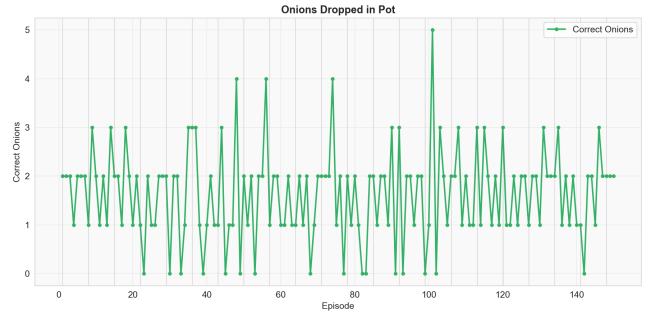


Figure 11: Amount of Onions Correctly Dropped

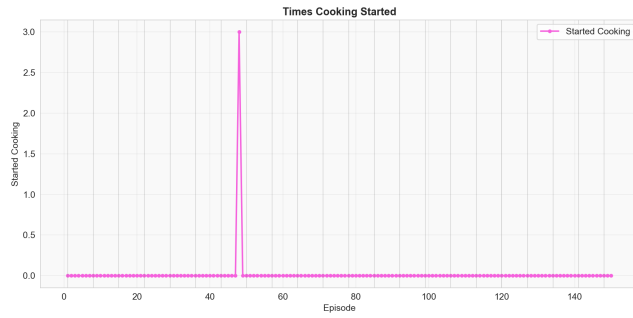


Figure 12: Times Pot Turned to Cooking

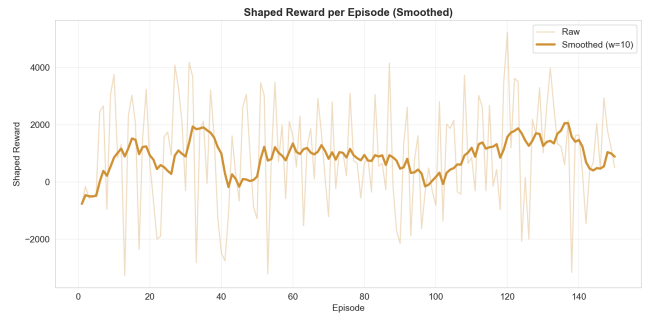


Figure 13: Smoothed Shaped Rewards

Results from the coordination ring further illustrate the underlying issues found in the cramped room. Not a single dish made and a lot less instances of the pot turned to cooking after being filled with three onions. The suspicions of a reward exploitation is further solidified with the smoothed reward shaping steadily inclining despite the other metrics. The fixation for onions and dishes could be from too heavy of a stacking penalty given when incorrectly placed or picked up.

5.4 Counter Circuit

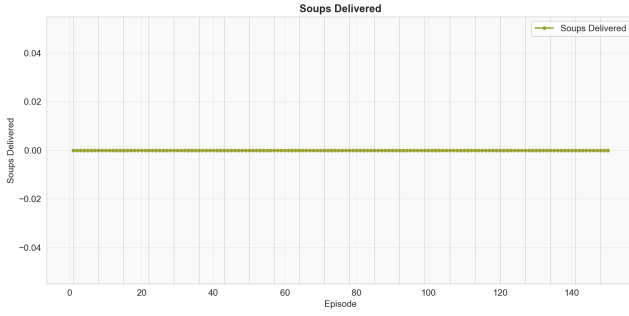


Figure 14: Environment Rewards

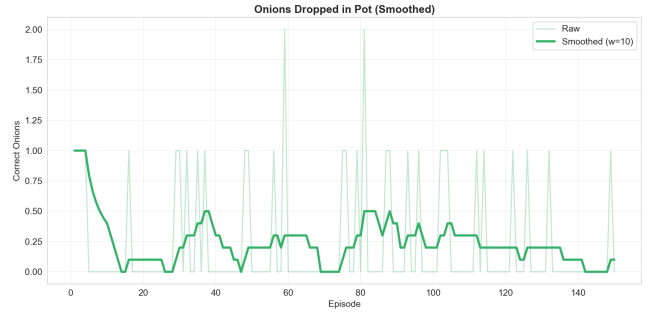


Figure 15: Onions Dropped

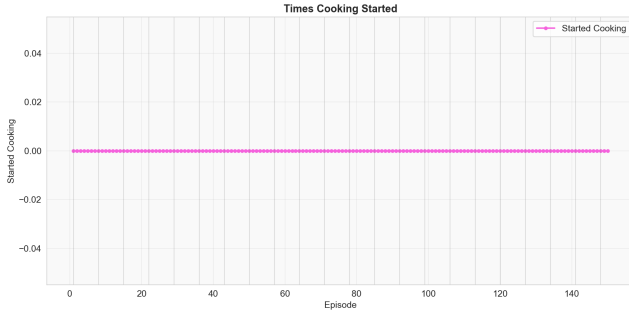


Figure 16: Pot Cooking

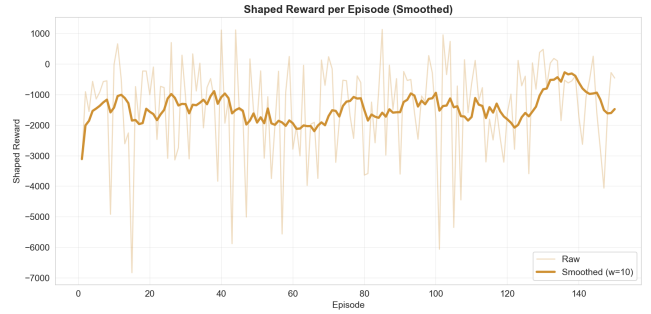


Figure 17: Shaped Rewards

Similar to the results found in the Coordination Ring, the Counter Circuit only accentuated the problems eluded to in the Cramped Room testing. Again, no soups were successfully served. Pit falls were fallen deeper and the exploitation of the imbalanced reward shaping worsened. When visualized into a gif, the agents ceased to move, as if in a paralyzed state, due to an intensely compromising logic predicament it built between penalties and rewards.

When reflecting on each map and their increasingly more difficult details, the QMIX did struggle more with the double charging of actions when facilitating reward sharing for each of the agents. Pit falls were easily found due to the struggle of assymetric observations by Agent 0, inhibiting all efforts to truly locate the hidden true issue. For reward imbalancing issues, efforts were made to compensate but unfortunately did not impact heavily enough to remedy some of these issues.

6 Conclusion

Despite extensive tuning, our QMIX agents failed to reach the target of ≥ 7 soups per episode across all three Overcooked layouts, averaging only 3.2 in `cramped_room`, 0.8 in `coordination_ring`, and 0.0 in `counter_circuit_order` during 150-episode evaluation. This shortfall—while disappointing—illuminates deeper truths about multi-agent reinforcement learning than any success metric could.

The core failure was reward shaping fragility. As documented in Section ??, dense positional bonuses and event-based signals initially accelerated pipeline progress, but their sensitivity to scaling led to catastrophic exploits: agents learned to serve incomplete soup for a net +300 gain, or froze in "paralysis by analysis" under over-penalization. Even with QMIX's monotonic value decomposition, double-charging a single action corrupted the shared TD signal, inflating Q -values and destabilizing training. These bugs were not edge cases—they were systematic, arising from assymmetric observations (POS_OBJ only for Agent 0) and insufficient credit assignment to Agent 1.

Yet, the episodic memory dictionary emerged as a quiet hero: by blocking redundant actions and capping penalty stacking, it prevented reward gaming and enabled Agent 0 to anticipate Agent 1's pot-blocking behavior—fetching dishes preemptively. This emergent coordination, though fragile, reveals QMIX's strength in loose coupling but its limitation in tight synchronization tasks like `counter_circuit`.

6.1 Lessons Learned

- **Dense rewards are double-edged:** They guide, but must be strictly bounded ($\leq 10\%$ of environmental signal) and ablated early.
- **Negative shaping induces fear:** Over-penalization causes policy collapse; agents learn avoidance, not progress.
- **QMIX excels in role specialization, not forced collaboration:** Its mixing network enables anticipatory behavior but assumes symmetric observability—a flaw we only partially patched with `DIST_OTHER`.
- **Bugs in reward logic are MARL poison:** A single double-charge floods the joint Q -function with agent-specific noise, breaking monotonicity.

6.2 Possible Expansions

- **Switch to MAPPO** with symmetric observations and centralized critic to better handle tight coordination.
- **Manual Reward shaping vs environment-only** training to quantify the true cost of manual intervention.
- **Introduce tomatoes** and recipe-specific goals to test curriculum learning and communication channels [?].
- **Test with more episodes** since only 5000 was not enough to obtain a full convergence for Overcooked.

This project taught me that MARL is less about algorithms and more about engineering trust—between agents, between reward and behavior, and between implementation and intent. The kitchen may not have served seven soups, but it served a deeper lesson: robustness begins where shaping ends.

References

1. T. Rashid, M. Samvelyan, C. Schroeder de Witt, *et al.*, “Weighted QMIX: Expanding monotonic value function factorisation,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
2. R. Lowe, Y. Wu, A. Tamar, *et al.*, “Multi-agent actor-critic for mixed cooperative-competitive environments,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
3. J. Foerster, I. A. Assael, N. de Freitas, and S. Whiteson, “Learning to communicate with deep multi-agent reinforcement learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.